1606. Find Servers That Handled Most Number of Requests

Daily Question - November 9, 2026

Problem Statement

You have k servers numbered from 0 to k - 1 that handle multiple requests simultaneously. Each server has infinite computational capacity but cannot handle more than one request at a time.

For the ith request:

If all servers are busy, the request is dropped. If the (i % k)th server is available, assign the request to that server. Otherwise, assign it to the next available server, wrapping around from server 0 if necessary.

You are given a strictly increasing array arrival, where arrival[i] is the arrival time of the ith request, and an array load, where load[i] is the time required to complete the ith request. Return the IDs of all servers that successfully handled the greatest number of requests, in any order.

Examples

Example 1: k = 3, arrival = [1,2,3,4,5], load = [5,2,3,3,3]. Output: [1]. Server 1 handles requests 1 and 3; servers 0 and 2 each handle one request; request 4 is dropped.

Example 2: k = 3, arrival = [1,2,3,4], load = [1,2,1,2]. Output: [0]. Server 0 handles requests 0 and 3; servers 1 and 2 each handle one request.

Example 3: k = 3, arrival = [1,2,3], load = [10,12,11]. Output: [0,1,2]. Each server handles one request.

Constraints

1 <= k <= 100000; 1 <= arrival.length, load.length <= 100000; arrival.length == load.length; 1 <= arrival[i], load[i] <= 1000000; arrival is strictly increasing.

Solution: TreeSet + Min-Heap

Maintain a TreeSet of currently available server IDs and a min-heap of busy servers ordered by completion time. For request i, first return every busy server whose completion time is at most arrival[i] to the available set.

The preferred starting server is i % k. Use available.ceiling(i % k) to find the first available server with ID at least i % k. If no such server exists, wrap around using available.first(). If the available set is empty, drop the request.

Assign the chosen server, increment its handled count, remove it from the available set, and insert [arrival[i] + load[i], server] into the busy heap. Finally, return every server whose handled count equals the maximum.

Complete Java Solution

```
import java.util.*;

class Solution {
```

```
    public List<Integer> busiestServers(int k, int[] arrival, int[] load) {
        int[] handled = new int[k];

        TreeSet<Integer> available = new TreeSet<>();
        for (int server = 0; server < k; server++) {
            available.add(server);
        }

        PriorityQueue<int[]> busy = new PriorityQueue<>(
                Comparator.comparingInt(request -> request[0])
        );

        for (int request = 0; request < arrival.length; request++) {
            int startTime = arrival[request];
            int endTime = startTime + load[request];

            while (!busy.isEmpty() && busy.peek()[0] <= startTime) {
                available.add(busy.poll()[1]);
            }

            if (available.isEmpty()) {
                continue;
            }

            int preferredServer = request % k;
            Integer server = available.ceiling(preferredServer);

            if (server == null) {
                server = available.first();
            }

            available.remove(server);
            handled[server]++;
            busy.offer(new int[] {endTime, server});
        }

        int maximum = 0;
        for (int count : handled) {
            maximum = Math.max(maximum, count);
        }

        List<Integer> answer = new ArrayList<>();
        for (int server = 0; server < k; server++) {
            if (handled[server] == maximum) {
                answer.add(server);
            }
        }

        return answer;
    }
}
```

Correctness

The busy heap correctly identifies all servers whose work has completed by the current request arrival time. The TreeSet maintains available server IDs in sorted order. For request i, ceiling(i % k) gives the first eligible server at or after the preferred position; if none exists, first() correctly wraps around to the smallest available ID.

If the available set is empty, all servers are busy and the request is dropped, exactly as required. Otherwise, the selected server is removed, its successful-request count is incremented, and it becomes busy until arrival[i] + load[i]. Therefore, handled[server] is exactly the number of requests successfully processed by that server. Returning all servers with the maximum count gives the required answer.

Complexity

Let n = arrival.length. Time is O(n log k): each request performs at most one TreeSet lookup and removal, plus one heap insertion. Each server is released at most once per completed request, so the total heap operations are O(n log k). Space is O(k) for the available set, busy heap, and handled-count array.

Implementation Notes

Use Integer for the result of TreeSet.ceiling because it may return null when no available server has an ID at least i % k. The busy heap uses [completionTime, serverId]; completion time is arrival[i] + load[i]. Arrival times are strictly increasing, but completion times need not be, so a heap is required rather than a simple queue.